



INSTITUTO FEDERAL

Santa Catarina

Câmpus São José

Aluno: Igor da Silva

Data: 12/05/2026

Disciplina: Sistemas Operacionais - 2026.1

Laboratório 02 - Coordenação

Escreva um documento com as respostas e capturas de tela dos códigos mostrando seu funcionamento correto.

1. Além do conceito de Monitor (com synchronized), a linguagem Java também oferece implementações explícitas de Mutex e Semáforo através do pacote `java.util.concurrent`. Reescreva, em Java, os exemplos de semáforos e mutex originalmente implementados em C.

- Em Java, o Mutex é implementado pela classe `ReentrantLock`. Ele funciona exatamente como o `pthread_mutex_t` do C: você precisa dar o `lock()` manualmente e, obrigatoriamente, o `unlock()`.
- O Java utiliza a classe `Semaphore`. Você define o número de "permissões"(vagas) no construtor.

2. Implemente semáforos, mutex e monitores para o caso do conta bancaria.

- Utilize `Thread.sleep(0)` para ter troca de contexto no caso de monitores.

3. Para o exemplo da pizzeria em monitores, melhore o exemplo com a possibilidade de existir um buffer/fila, isto é, O cozinheiro pode ser mais rápido e "estocar"até 2 pizzas.

- Faça testes em `Thread.sleep(0)` em relação aos tempos de preparação e entrega da pizza (ex: cozinheiro lento, entregador fica esperando).
- Utilize `notifyAll()` para garantir que todos os interessados acordem
- Utilize `Queue` e seus métodos `add`, `size`, `isEmpty` e `poll`

4. Procure as limitações dos semáforos e o que essas elas podem provocar.

Questão 1)

Escolhi o problema do contador. A seguir, segue-se a captura de tela do funcionamento tanto da solução com semáforo, quanto com mutex, respectivamente.

Captura do funcionamento com semáforo - questão do contador.

```
J SemaforoContador.java x
lab-coordenacao-02 > item1 > J SemaforoContador.java > SemaforoContador > main(String[])
1 // Semaforo: https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/Semaphore.html
2 // Thread: https://docs.oracle.com/javase/8/docs/api/java/lang/Thread.html
3 // Fiz o código m5-semaforo (as threads somam a variável sum).
4
5 import java.util.ArrayList;
6 import java.util.concurrent.Semaphore;
7
8 public class SemaforoContador {
9
10     static final int NUM_THREADS = 100;
11     static final int NUM_STEPS = 100000;
12     static final int vagas = 1;
13
14     static int sum = 0;
15     static Semaphore s = new Semaphore(vagas);
16
17     private static void somar(){
18
19         for(int i = 0; i < NUM_STEPS; i++){
20
21             try{
22                 s.acquire();
23             } catch (InterruptedException e){
24                 System.out.println(e);
25             }
26
27             sum++;
28
29             if(sum % NUM_STEPS == 0)
30                 System.out.println("Sum: " + sum + "\n");
31
32             s.release();
33         }
34     }
35 }

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL

Sum: 9600000
Sum: 9700000
Sum: 9800000
Sum: 9900000
Sum: 10000000

Valor esperado: 10000000
Valor obtido: 10000000
grupofv: Sistemas-Operacionais$
```

Captura do funcionamento com mutex - questão do contador.

```
J MutexContador.java X
lab-coordenacao-02 > item1 > J MutexContador.java > MutexContador > somar()
1  import java.util.ArrayList;
2  import java.util.concurrent.locks.ReentrantLock;
3
4  public class MutexContador{
5
6      static ReentrantLock cadeado = new ReentrantLock();
7      static final int NUM_THREADS = 100;
8      static final int NUM_STEPS = 100000;
9      static int sum = 0;
10
11     public static void somar(){
12
13         cadeado.lock();
14
15         for(int i = 0; i < NUM_STEPS; i++){
16             sum++;
17         }
18         cadeado.unlock();
19     }
20 }
21
22 Run | Debug
23 public static void main(String args[]){
24
25     ArrayList<Thread> t = new ArrayList<Thread>(NUM_THREADS);
26     long expected = (long) NUM_STEPS * NUM_THREADS;
27
28     // agora eu vou criar as threads e depois inicia-las
29     for(int i = 0; i < NUM_THREADS; i++){
30
31         Thread ti = new Thread(()->{
32
33             somar();
34         });
35     }
36 }
37
38 PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL
39
40 grupofv: Sistemas-Operacionais$ /usr/bin/env /usr/lib/jvm/jdk-17-oracle
41 kspaceStorage/d4ef572b63808c4384b6b1f25256633f/redhat.java/jdt_ws/Sistem
42 Valor esperado: 10000000
43 Valor obtido: 10000000
44 grupofv: Sistemas-Operacionais$
```

Questão 2)

A seguir, seguem-se as capturas de tela das três soluções, respectivamente, com semáforo, mutex e monitor do caso da conta bancária.

Captura do funcionamento com semáforo - questão da conta bancária.

```
ContaBancariaSemaforo.java X
lab-coordenacao-02 > item2 > J ContaBancariaSemaforo.java > {} item2
1 package item2;
2 import java.util.concurrent.*;
3
4 public class ContaBancariaSemaforo{
5
6     static final int PERMISSOES = 1;
7     static final int VALOR_T1 = 30;
8     static final int VALOR_T2 = 50;
9
10    static int conta = 100;
11    static Semaphore s = new Semaphore(PERMISSOES);
12
13    public static void atualiza_conta(int valor){
14
15        try{
16            s.acquire();
17        } catch (InterruptedException e){
18            System.err.println(e);
19        }
20
21        conta += valor;
22
23        s.release();
24    }
25 }
```

PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL

```
grupofv: Sistemas-Operacionais$ /usr/bin/env /usr/lib/jvm/j
kspaceStorage/d4ef572b63808c4384b6b1f25256633f/redhat.java/j
Valor final esperado: 180
Valor final obtido: 180
grupofv: Sistemas-Operacionais$
```

Captura do funcionamento com mutex - questão da conta bancária.

```
J ContaBancariaMutex.java 1 x
lab-coordenacao-02 > item2 > J ContaBancariaMutex.java > {} item2
1  package item2;
2
3  import java.util.concurrent.*;
4  import java.util.concurrent.locks.ReentrantLock;
5
6  public class ContaBancariaMutex{
7
8      static final int VALOR_T1 = 30;
9      static final int VALOR_T2 = 50;
10
11     static int conta = 100;
12     static ReentrantLock cadeado = new ReentrantLock();
13
14     public static void atualiza_conta(int valor){
15
16         cadeado.lock();
17
18         conta += valor;
19
20         cadeado.unlock();
21
22     }
23
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL
grupofv: Sistemas-Operacionais$ /usr/bin/env /usr/lib/jvm/jdk
kspacStorage/d4ef572b63808c4384b6b1f25256633f/redhat.java/jdt
Valor final esperado: 180
Valor final obtido: 180
grupofv: Sistemas-Operacionais$
```

Captura do funcionamento com monitor - questão da conta bancária.

```
J SaldoComMonitor.java x
lab-coordenacao-02 > item2 > J SaldoComMonitor.java > SaldoComMonitor > getSaldo()
1 package item2;
2
3 class SaldoComMonitor{
4
5     private int saldo = 100;
6
7     public synchronized void atualiza_conta(int valor){
8
9         this.saldo += valor;
10
11     }
12
13     public synchronized int getSaldo(){
14
15         return this.saldo;
16
17     }
18
19 }
1
```

```
J ContaBancariaMonitor.java x
lab-coordenacao-02 > item2 > J ContaBancariaMonitor.java > ContaBancariaMonitor
1 package item2;
2
3 public class ContaBancariaMonitor{
4
5     static final int VALOR_T1 = 30;
6     static final int VALOR_T2 = 50;
7
8     static SaldoComMonitor saldo = new SaldoComMonitor();
9
10
11     Run | Debug
12     public static void main(String args[]){
13
14         int expected = saldo.getSaldo() + VALOR_T1 + VALOR_T2;
15
16         Thread t1 = new Thread()->{
17
18             try{
19                 saldo.atualiza_conta(VALOR_T1);
20                 Thread.sleep(millis: 0);
21             }catch(InterruptedExce... e){
22                 System.err.println(e);
23             }
24         }
25     }
2
```

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL
grupofv: Sistemas-Operacionais$ /usr/bin/env /usr/lib/jvm/jdk-17-oracle-x64/bin/java -XX:+ShowCodeDetailsInExceptionMessages -cp /home/grupofv/.cont
kspaceStorage/d4ef572b63808c4384b6b1f25256633f/redhat.java/jdt_ws/Sistemas-Operacionais_379bdd79/bin item2.ContaBancariaMonitor
Valor final esperado: 180
Valor final obtido: 180
grupofv: Sistemas-Operacionais$
```

- 1- Classe criada para o monitor;
- 2- Código implementado.

Questão 3)

Nesta questão, criei a fila no arquivo da Classe MesaPizza. Testei o código para os casos:

1. cozinheiro rápido e entregador lento => a fila enchia rapidamente e o cozinheiro aguardava o entregador buscar as pizzas.
2. cozinheiro lento e entregador rápido => a fila não enchia e o entregador aguardava o cozinheiro preparar a pizza;
3. ambos em sincronia (Thread.sleep(0)) não gerava ociosidade.

A seguir, seguem-se as capturas de telas para os respectivos casos.

Caso 1 - fila de pizzas encheu e cozinheiro ocioso.

```
J MesaPizza.java M X J PizzaMonitor.java M
lab-coordenacao-02 > item3 > J MesaPizza.java > MesaPizza > colocarPizza(String)
6 class MesaPizza {
12 public synchronized void colocarPizza(String nomePizza) throws InterruptedException {
13
14     // Se a fila encheu a capacidade, o cozinheiro vai descansar
15     while (pizzas_prontas.size() >= MAX_PIZZAS) {
16         wait();
17         System.out.println(x: "Cozinheiro ocioso... ZZZ");
18     }
19
20     // se a fila ainda nao atingiu o limite, dá para cozinhar mais pizzas
21     pizzas_prontas.add(nomePizza);
22     // Avisa o entregador que tem pizza pronta
23     notifyAll();
24     System.out.println("Cozinheiro colocou a pizza de: " + nomePizza);
25 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
aluno: Sistemas-Operacionais$ cd /home/aluno/Documentos/Sistemas-Operacionais ; /usr/bin/env /usr/lib
XX:+ShowCodeDetailsInExceptionMessages -cp /home/aluno/.config/Code/User/workspaceStorage/59ea607a51a
ws/Sistemas-Operacionais_35120f84/bin item3.PizzaMonitor
Cozinheiro colocou a pizza de: Mussarela
Entregador levou a pizza de: Mussarela
Cozinheiro colocou a pizza de: Mussarela
Cozinheiro colocou a pizza de: Calabresa
Cozinheiro colocou a pizza de: Mussarela
Cozinheiro colocou a pizza de: Calabresa
Cozinheiro colocou a pizza de: Calabresa
Cozinheiro colocou a pizza de: Mussarela
Cozinheiro colocou a pizza de: Calabresa
Cozinheiro colocou a pizza de: Calabresa
Cozinheiro colocou a pizza de: Mussarela
Cozinheiro colocou a pizza de: Portuguesa
Entregador levou a pizza de: Mussarela
Cozinheiro ocioso... ZZZ
Cozinheiro colocou a pizza de: Portuguesa
Entregador levou a pizza de: Calabresa
Cozinheiro ocioso... ZZZ
Cozinheiro colocou a pizza de: Portuguesa
Entregador levou a pizza de: Mussarela
Cozinheiro ocioso... ZZZ
Cozinheiro colocou a pizza de: Mussarela
Entregador levou a pizza de: Calabresa
Cozinheiro ocioso... ZZZ
Cozinheiro colocou a pizza de: Mussarela
aluno: Sistemas-Operacionais$
```

Caso 2 - entregador ocioso.

```
J MesaPizza.java M    J PizzaMonitor.java M X
lab-coordenacao-02 > item3 > J PizzaMonitor.java > PizzaMonitor > main(String[])
5  public class PizzaMonitor{
7      public static void main(String[] args) {
43         while(true){
44             mesa.retirarPizza();
45             Thread.sleep(10); // Demora para entregar
46         }
47     } catch (InterruptedException e) {}
48     }.start();
49 }
50
51
52
53
54
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
aluno: Sistemas-Operacionais$ cd /home/aluno/Documentos/Sistemas-Operacionais ; /u
XX:+ShowCodeDetailsInExceptionMessages -cp /home/aluno/.config/Code/User/workspaces
ws/Sistemas-Operacionais_35120f84/bin item3.PizzaMonitor
Cozinheiro colocou a pizza de: Calabresa
Entregador levou a pizza de: Calabresa
Cozinheiro colocou a pizza de: Calabresa
Entregador ocioso... ZZZ
Entregador levou a pizza de: Calabresa
Cozinheiro colocou a pizza de: Mussarela
Entregador ocioso... ZZZ
Entregador levou a pizza de: Mussarela
Cozinheiro colocou a pizza de: Mussarela
Entregador ocioso... ZZZ
Entregador levou a pizza de: Mussarela
Cozinheiro colocou a pizza de: Mussarela
Entregador ocioso... ZZZ
Entregador levou a pizza de: Mussarela
Cozinheiro colocou a pizza de: Calabresa
Entregador ocioso... ZZZ
Entregador levou a pizza de: Calabresa
Cozinheiro colocou a pizza de: Calabresa
Entregador ocioso... ZZZ
Entregador levou a pizza de: Calabresa
^Caluno: Sistemas-Operacionais$
```


Caso 3 - sincronia: ambos com Thread.sleep(0).

```
J MesaPizza.java M J PizzaMonitor.java M X
lab-coordenacao-02 > item3 > J PizzaMonitor.java > PizzaMonitor > main(String[])
5 public class PizzaMonitor{
7     public static void main(String[] args) {
26         // Quando o entregador removiu uma pizza, o cozinheiro preparava outra e a adicionava na fila.
27
28         // Quando coloquei ambos em 0, nunca enchia a fila
29
30         // Quando coloquei o cozinheiro com 1000 ms e o entregador com 0 ms, nunca enchia a fila: assim
31         // que o cozinheiro preparava uma pizza, o entregador já a entregava.
32
33     }
34
35     } catch (InterruptedException e) {}
36
37     }.start();
38
39     // Thread do Entregador
40     new Thread(() -> {

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Cozinheiro colocou a pizza de: Mussarela
Entregador levou a pizza de: Mussarela
Cozinheiro colocou a pizza de: Portuguesa
Entregador levou a pizza de: Portuguesa
Cozinheiro colocou a pizza de: Mussarela
Entregador levou a pizza de: Mussarela
Cozinheiro colocou a pizza de: Portuguesa
Entregador levou a pizza de: Portuguesa
Cozinheiro colocou a pizza de: Calabresa
Entregador levou a pizza de: Mussarela
Cozinheiro colocou a pizza de: Calabresa
Entregador levou a pizza de: Portuguesa
Cozinheiro colocou a pizza de: Calabresa
Entregador levou a pizza de: Calabresa
Cozinheiro colocou a pizza de: Calabresa
Entregador levou a pizza de: Calabresa
Cozinheiro colocou a pizza de: Mussarela
Entregador levou a pizza de: Calabresa
Cozinheiro colocou a pizza de: Portuguesa
Entregador levou a pizza de: Calabresa
Cozinheiro colocou a pizza de: Portuguesa
Entregador levou a pizza de: Mussarela
Cozinheiro colocou a pizza de: Mussarela
Entregador levou a pizza de: Portuguesa
Cozinheiro colocou a pizza de: Portuguesa
Entregador levou a pizza de: Portuguesa
Cozinheiro colocou a pizza de: Portuguesa
aluno: Sistemas-Operacionais$
```

> Conclusões:

- Quando coloquei o cozinheiro com 10 ms e o entregador com 1000 ms, a fila encheu as 10 pizzas, o cozinheiro dormiu e aguardou o entregador retirar pelo menos uma pizza.
- Quando o entregador removia uma pizza, o cozinheiro preparava outra e a adicionava na fila.
- Quando coloquei ambos em 0, nunca enchia a fila

Questão 4)

Procure as limitações dos semáforos e o que essas elas podem provocar.

- O semáforo pode deixar a execução lento, por causa do *overhead* para sincronizar com o sistema operacional, pode causar comboios.
-
- Pode causar espera ocupada, utilizando a CPU para verificar o valor atual do semáforo.
-
- Pode causar *starvation*. Utilizando o exemplo do estacionamento com 5 vagas: se nenhum dos 5 carros estacionados deixarem o estacionamento, o sexto carro (na espera) e os demais, nunca conseguirão estacionar.